

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Vulnerability Detection

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation		
Student Name:	Adhithyan S	Reg. No.:	192421103
Section / Batch:	BE CYB SEC / 3rd Year	Date:	

Instructions to Students

- Perform vulnerability detection and classify the identified vulnerabilities based on their severity levels.
- Conduct severity analysis using appropriate metrics such as CVSS and document the impact of each vulnerability.
- Design and simulate a Security Verification Workflow covering detection, remediation, re-testing, and verification.
- Evaluate security and system performance before and after mitigation, and include relevant results, screenshots, and observations.

Question Type	Focus Area	Questions	Marks
Vulnerability Detection	Conceptual Understanding	Q1	Q1 –100
GRAND TOTAL:			100 Marks

Code of Conduct

*I **Adhithyan S (192421102)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Adhithyan S

Q1 — Vulnerability Detection, Severity Analysis & Security Verification Workflow

Detection → CVSS Severity Analysis → Remediation → Re-testing → Verification → Performance Evaluation

1. Objective & Problem Understanding

This assessment simulates a complete vulnerability management lifecycle for a sample web application. It covers detecting vulnerabilities, scoring their severity using the Common Vulnerability Scoring System (CVSS), designing a Security Verification Workflow (Detection → Remediation → Re-testing → Verification), and evaluating system/security performance before and after mitigation.

Scope of the solution:

- Detect vulnerabilities in a sample target application using SAST/DAST/SCA style scanning.
- Classify each vulnerability by severity (Critical / High / Medium / Low) using CVSS v3.1.
- Simulate a four-stage Security Verification Workflow with defined entry/exit criteria per stage.
- Remediate the highest-risk findings and re-test to confirm closure.
- Compare security posture and system performance metrics before vs. after mitigation.

2. Tools & Environment

Category	Tool	Purpose
Target Application	OWASP Juice Shop (sample)	Intentionally vulnerable reference app
DAST	OWASP ZAP	Runtime vulnerability detection
SCA	pip-audit	Detects vulnerable third-party dependencies
Static Analysis	Bandit / Semgrep	Detects insecure code patterns
Severity Scoring	CVSS v3.1 Calculator (FIRST.org)	Computes base score & severity rating
Load Testing	Apache JMeter	Measures performance before/after mitigation

3. Vulnerability Detection — Findings

The table below lists vulnerabilities identified across DAST, SCA, and static analysis scans of the target application.

ID	Vulnerability	Category	Location
V-01	SQL Injection in login form	DAST	/rest/user/login
V-02	Reflected Cross-Site Scripting (XSS)	DAST	/rest/products/search
V-03	PyYAML unsafe deserialization (CVE-2020-14343)	SCA	requirements.txt
V-04	Hard-coded API secret in source code	SAST	config.py
V-05	Missing security headers (CSP, X-Frame-Options)	DAST	All pages
V-06	Session cookie without Secure/HttpOnly flag	DAST	/rest/user/login

4. Severity Analysis using CVSS v3.1

Each finding was scored using the CVSS v3.1 base metrics (Attack Vector, Attack Complexity, Privileges Required, User Interaction, Scope, Confidentiality, Integrity, Availability impact).

ID	CVSS Vector	Base Score	Severity
V-01	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Critical
V-03	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Critical
V-02	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N	6.1	Medium
V-04	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	7.5	High
V-06	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Medium
V-05	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N	4.3	Low

Impact summary: V-01 and V-03 are Critical — both allow full compromise of confidentiality, integrity, and availability with no authentication or user interaction required, and are prioritized first for remediation. V-04 is High due to full confidentiality loss from secret exposure. V-02 and V-06 are Medium risk. V-05 is Low but still remediated as defense-in-depth.

5. Security Verification Workflow (Simulation)

The workflow below governs how every finding above moves from discovery to closure. It has four gated stages; a finding cannot advance until its exit criteria are met.

Stage	Activities	Exit Criteria
1. Detection	Run DAST (ZAP), SCA (pip-audit) and SAST (Bandit) scans; log findings with CVSS score	All findings logged with severity assigned
2. Remediation	Developer applies fix (code change / dependency upgrade / config hardening)	Fix merged; pipeline SCA/SAST gate passes
3. Re-testing	Re-run the same scanner/rule that originally flagged the issue against the fixed build	Original finding no longer detected
4. Verification	Security reviewer manually confirms the fix and closes the ticket; regression scan run on full app	Finding marked 'Verified Closed'; no new regressions introduced

6. Remediation Implementation

6.1 V-01 — Fix SQL Injection (parameterized query)

<pre># Before (vulnerable): string concatenation query = "SELECT * FROM users WHERE email='" + email + "'" # After (fixed): parameterized query cursor.execute("SELECT * FROM users WHERE email = ?", (email,))</pre>
--

6.2 V-03 — Fix unsafe YAML deserialization

```
# Before (vulnerable)

data = yaml.load(user_input)


# After (fixed)

data = yaml.safe_load(user_input)


# Also bump the dependency:

pyyaml==6.0.1    # was 5.3
```

6.3 V-04 — Remove hard-coded secret

```
# Before (vulnerable)

API_KEY = "sk_live_51Hc...redacted..."


# After (fixed) – load from environment / secret manager

import os

API_KEY = os.environ["API_KEY"]
```

6.4 V-05 / V-06 — Security headers & cookie flags

```

from flask_talisman import Talisman

Talisman(app, content_security_policy={'default-src': "'self'"})


app.config.update(

    SESSION_COOKIE_SECURE=True,

    SESSION_COOKIE_HTTPONLY=True,

    SESSION_COOKIE_SAMESITE='Strict'

)

```

7. Re-testing & Verification Results

ID	Severity	Status Before	Re-test Result	Verification
V-01	Critical	Open	No longer exploitable — parameterized query confirmed	Verified Closed
V-03	Critical	Open	pip-audit reports 0 known CVEs on pyyaml 6.0.1	Verified Closed
V-04	High	Open	Bandit scan clean — no hard-coded secrets detected	Verified Closed
V-02	Medium	Open	ZAP re-scan: input now output-encoded, XSS not triggered	Verified Closed
V-06	Medium	Open	Cookie inspected in DevTools — Secure & HttpOnly present	Verified Closed
V-05	Low	Open	Response headers include CSP, X-Frame-Options: DENY	Verified Closed

8. Performance Evaluation — Before vs. After Mitigation

Load was simulated with Apache JMeter (50 concurrent users, 5-minute steady-state) against the application before and after remediation, alongside a security posture score.

Metric	Before Mitigation	After Mitigation	Change
Avg. Response Time	412 ms	438 ms	+26 ms (~6% slower)
Throughput	121 req/s	116 req/s	-5 req/s (~4% lower)
Error Rate	2.1%	0.0%	-2.1% (improved)
Open Critical/High Findings	3	0	-3 (resolved)
Overall CVSS Risk Exposure	9.8 (Critical)	0.0 (None open)	Fully mitigated
Security Headers Present	0 / 3	3 / 3	+3 (hardened)

Observation: Remediation introduced a small, acceptable overhead (~6% latency, ~4% throughput) from added input validation, parameterized queries, and security middleware. This trade-off is justified given that all Critical and High severity vulnerabilities were closed and the error rate actually improved, since malformed/malicious requests are now rejected safely instead of causing server-side exceptions.

9. Conclusion

This simulation walked a set of six vulnerabilities through a complete lifecycle: detection via DAST/SCA/SAST tooling, CVSS-based severity scoring, a gated four-stage verification workflow, targeted remediation, and re-test/verification. Post-mitigation evaluation confirmed all Critical and High risk findings were closed with only a minor, acceptable performance overhead — demonstrating that security hardening and system performance can be balanced effectively within a DevSecOps pipeline.

Note: The findings, CVSS scores, and performance figures above are simulated for demonstration on a reference vulnerable application. When submitting, replace them with your own scan outputs/screenshots (ZAP, pip-audit, Bandit, JMeter) and CVSS calculator results as required by the assignment instructions.